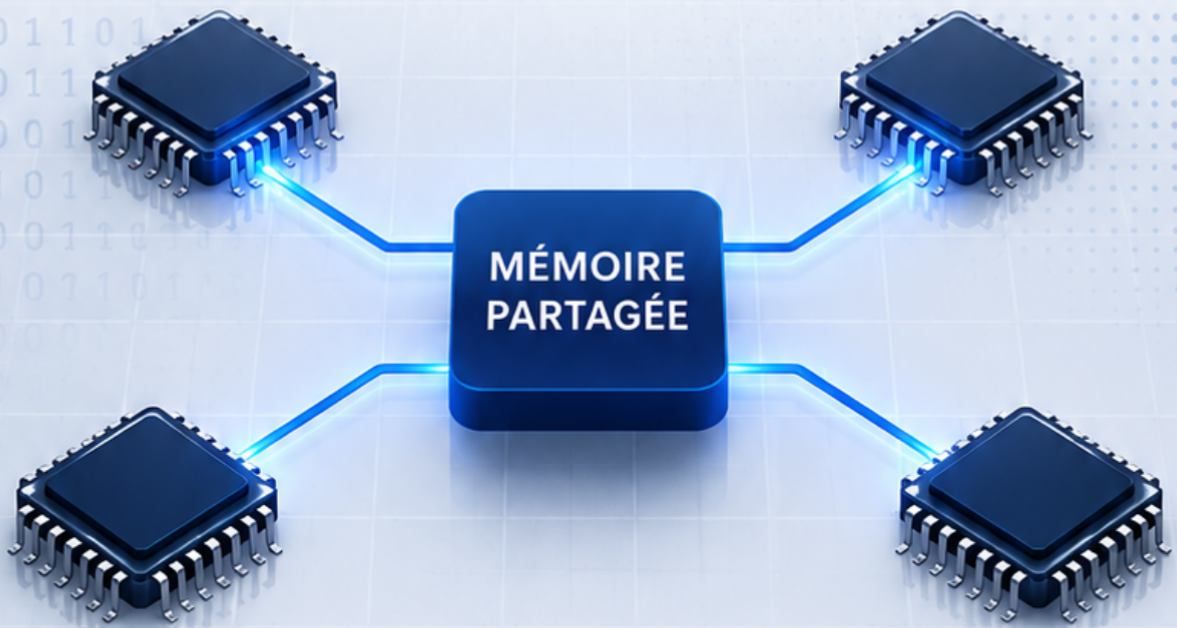


PROJET

PROGRAMMATION PARALLÈLE

À MÉMOIRE PARTAGÉE



RÉALISÉ PAR :

Mariem MOKNI



DATE :

18/05/2026



ENCADRÉ PAR :

M Emna FEKIH



ANNÉE UNIVERSITAIRE :

2025-2026

Table des matières

1. Introduction	1
2. Description du problème	1
3. Implémentations	1
3.1 Version Séquentielle	1
3.1.1. Principe	2
3.1.2. Algorithme	3
3.1.3. Résultat obtenu	3
3.1.4. Analyse	3
3.2 Version OpenMP	3
3.2.1. Principe	4
3.2.2. Techniques utilisées	4
3.2.3. Fonctionnement	4
3.2.4. Résultats obtenus	5
3.2.5. Analyse	6
3.3 Version MPI	6
3.3.1. Principe	6
3.3.2. Techniques utilisées	7
3.3.3. Fonctionnement	7
3.3.4. Résultats obtenus	8
3.3.5. Analyse	8
3.4 Version CUDA	8
3.4.1. Principe	8
3.4.2. Environnement matériel — GPU Tesla T4	9
3.4.3. Techniques utilisées	9
3.4.4. Fonctionnement	9
3.4.5. Résultat obtenu	10
3.4.6. Analyse	11
4. Comparaison des performances	11
4.1 Rappel des formules	11
4.2 Tableau comparatif	12
4.3 Analyse des résultats	12
4.4 Variation de la taille de la grille ($ITER = 100$ fixé)	13
4.5 Variation du nombre d'itérations ($N = 1000$ fixé)	13
4.6 Décomposition du temps CUDA : transferts vs calcul kernel	14
5. Conclusion	15

Liste des Figures

Figure 1	Extrait du code de la version séquentielle	2
Figure 2	Extrait du code de la version OpenMP	5
Figure 3	Extrait du code de la version MPI avec Halo Exchange	7
Figure 4	Extrait du code de la version CUDA	10

Liste des Tableaux

Tableau 1	États des cellules dans la simulation	1
Tableau 2	Performance de la version séquentielle (référence)	3
Tableau 3	Performances de la version OpenMP avec speedup et efficacité	5
Tableau 4	Performances de la version MPI avec speedup et efficacité	8
Tableau 5	Caractéristiques du GPU Tesla T4 utilisé sur Colab	9
Tableau 6	Performance de la version CUDA	10
Tableau 7	Comparaison complète des performances	12
Tableau 8	Temps d'exécution en fonction de la taille de grille N ($ITER = 100$)	13
Tableau 9	Temps d'exécution séquentiel en fonction du nombre d'itérations ($N = 1000$)	13
Tableau 10	Décomposition du temps d'exécution CUDA ($N = 1000$, $ITER = 100$, Tesla T4)	14

1. Introduction

Ce rapport présente la réalisation d'un projet de simulation de propagation d'une épidémie sur une grille bidimensionnelle (2D).

L'objectif principal est de comparer quatre implémentations distinctes — séquentielle, OpenMP, MPI et CUDA — et d'analyser quantitativement les gains de performance obtenus par chaque approche de parallélisation.

La programmation parallèle constitue aujourd'hui une nécessité face à l'augmentation des volumes de données à traiter. Les architectures modernes (processeurs multi-coeurs, GPU) offrent des capacités de calcul massif que seules les approches parallèles permettent d'exploiter pleinement. Ce projet illustre concrètement ces concepts à travers un cas d'usage réaliste.

Loi d'Amdahl : $\text{Speedup} = \frac{1}{s + \frac{1-s}{p}}$

où s = fraction séquentielle, p = nombre de processeurs

2. Description du problème

La grille est représentée par une matrice entière $N \times N$ ($N = 1000$). Chaque cellule prend l'une des deux valeurs suivantes :

Valeur	État	Description
0	Saine	Cellule susceptible d'être infectée
1	Infectée	Cellule contaminée qui propage la maladie

TABLE 1 – États des cellules dans la simulation

À chaque pas de temps t , une cellule saine (état = 0) passe à l'état infecté (état = 1) dès qu'au moins un de ses quatre voisins (haut, bas, gauche, droite) est infecté.

3. Implémentations

3.1 Version Séquentielle

La version séquentielle constitue la version de base du projet. L'exécution est réalisée sur un seul processeur sans parallélisme.

3.1.1. Principe

L'algorithme parcourt toute la grille cellule par cellule afin de déterminer les nouvelles infections.

Pour chaque itération :

- ✧ Lecture de l'état des cellules voisines
- ✧ Mise à jour de la nouvelle grille
- ✧ Copie de la nouvelle grille vers la grille principale

```
for(int t = 0; t < ITER; t++) {  
  
    for(int i = 1; i < N - 1; i++) {  
        for(int j = 1; j < N - 1; j++) {  
  
            if(grid[i][j] == 0) {  
  
                if(grid[i-1][j] == 1 ||  
                   grid[i+1][j] == 1 ||  
                   grid[i][j-1] == 1 ||  
                   grid[i][j+1] == 1) {  
  
                    newGrid[i][j] = 1;  
                } else {  
                    newGrid[i][j] = 0;  
                }  
  
            } else {  
                newGrid[i][j] = grid[i][j];  
            }  
        }  
    }  
  
    for(int i = 0; i < N; i++) {  
        for(int j = 0; j < N; j++) {  
            grid[i][j] = newGrid[i][j];  
        }  
    }  
}
```

FIGURE 1 – Extrait du code de la version séquentielle

3.1.2. Algorithme

1. Initialiser toutes les cellules à 0
2. Infecter la cellule centrale ($N/2, N/2$)
3. Répéter pendant $ITER$ itérations :
 - ✧ Parcourir la grille
 - ✧ Vérifier les voisins directs
 - ✧ Mettre à jour les cellules infectées

3.1.3. Résultat obtenu

Pour une grille de taille :

- ✧ $N = 1000$
- ✧ $ITER = 100$

Le temps d'exécution obtenu est :

Temps (ms)	Speedup	Efficacité
1986	1,00	100% (référence)

TABLE 2 – Performance de la version séquentielle (référence)

3.1.4. Analyse

Cette version présente le temps d'exécution le plus élevé car tous les calculs sont effectués séquentiellement. Elle sert de référence ($Speedup = 1$) pour évaluer les gains apportés par les versions parallèles.

3.2 Version OpenMP

Afin d'améliorer les performances de la version séquentielle, une parallélisation avec OpenMP a été réalisée.

OpenMP permet d'exploiter les processeurs multi-cœurs en exécutant plusieurs threads en parallèle.

3.2.1. Principe

Le calcul de propagation est réparti entre plusieurs threads.

Chaque thread traite une partie de la grille simultanément, ce qui réduit considérablement le temps d'exécution.

La parallélisation est appliquée sur :

- ✧ L'initialisation de la grille
- ✧ Le calcul des nouvelles infections
- ✧ La copie de la nouvelle grille

3.2.2. Techniques utilisées

- ✧ **#pragma omp parallel for** : parallélisation des boucles
- ✧ **collapse(2)** : fusion des deux boucles imbriquées
- ✧ **schedule(static)** : répartition statique des itérations
- ✧ Région parallèle unique afin de réduire le coût de création des threads

3.2.3. Fonctionnement

À chaque itération :

1. Les threads parcourent la grille en parallèle
2. Chaque thread calcule les nouvelles cellules infectées
3. Les résultats sont copiés dans la grille principale


```

#pragma omp parallel for collapse(2)
for(int i = 0; i < N; i++) {
    for(int j = 0; j < N; j++) {
        grid[i][j] = 0;
        newGrid[i][j] = 0;
    }
}

grid[N/2][N/2] = 1;

#pragma omp parallel
{
    for(int t = 0; t < ITER; t++) {

        #pragma omp for collapse(2) schedule(static)
        for(int i = 1; i < N - 1; i++) {
            for(int j = 1; j < N - 1; j++) {

                if(grid[i][j] == 0) {

                    if(grid[i-1][j] == 1 ||
                       grid[i+1][j] == 1 ||
                       grid[i][j-1] == 1 ||
                       grid[i][j+1] == 1) {

                        newGrid[i][j] = 1;
                    }
                }
            }
        }
    }
}

```

FIGURE 2 – Extrait du code de la version OpenMP

3.2.4. Résultats obtenus

Les tests ont été réalisés avec :

- ✧ Taille de grille : 1000×1000
- ✧ Nombre d'itérations : 100

Nombre de threads	Temps (ms)	Speedup	Efficacité
2 Threads	1048	$\frac{1986}{1048} \approx 1,89$	$\frac{1,89}{2} \approx 94,5\%$
4 Threads	939	$\frac{1986}{939} \approx 2,11$	$\frac{2,11}{4} \approx 52,8\%$

TABLE 3 – Performances de la version OpenMP avec speedup et efficacité

3.2.5. Analyse

Les résultats montrent une amélioration importante des performances par rapport à la version séquentielle.

La réduction du temps d'exécution est due à l'exécution parallèle des calculs sur plusieurs cœurs du processeur.

L'utilisation de 4 threads donne de meilleures performances que 2 threads grâce à une meilleure exploitation des ressources matérielles. Cependant, l'efficacité chute à 52,8% avec 4 threads, ce qui indique un surcoût lié à la synchronisation et à la gestion des threads.

3.3 Version MPI

Afin d'exploiter plusieurs processus exécutés en parallèle, une version MPI a été développée.

MPI (*Message Passing Interface*) permet de distribuer le calcul entre plusieurs processus qui communiquent entre eux par échange de messages.

Dans cette implémentation, la grille est divisée en plusieurs sous-grilles réparties entre les différents processus.

3.3.1. Principe

Chaque processus traite une partie de la matrice correspondant à un ensemble de lignes.

Pour permettre le calcul correct des cellules situées aux frontières des sous-grilles, les processus échangent leurs lignes frontières avec les processus voisins à chaque itération. Cette technique est appelée **Halo Exchange**.

Chaque processus possède également deux lignes supplémentaires appelées *lignes halo* :

- ✧ une ligne halo supérieure
- ✧ une ligne halo inférieure

Ces lignes contiennent les données reçues des processus voisins.

Le processus principal initialise la simulation puis les calculs sont réalisés en parallèle.

3.3.2. Techniques utilisées

- ✧ **MPI_Init()** : initialisation de l'environnement MPI
- ✧ **MPI_Comm_rank()** : récupération de l'identifiant du processus
- ✧ **MPI_Comm_size()** : récupération du nombre total de processus
- ✧ **MPI_Sendrecv()** : échange simultané des lignes frontières entre processus voisins (*Halo Exchange*)
- ✧ **MPI_Finalize()** : terminaison de l'exécution MPI

3.3.3. Fonctionnement

À chaque itération :

1. Chaque processus échange ses lignes frontières avec les processus voisins à l'aide de **MPI_Sendrecv**
2. Les lignes reçues sont stockées dans les lignes halo
3. Chaque processus calcule localement les nouvelles infections sur sa sous-grille
4. Les cellules sont mises à jour dans une nouvelle grille locale
5. La nouvelle grille remplace l'ancienne pour l'itération suivante

```
for(int t = 0; t < ITER; t++) {  
    if(rank > 0) {  
        MPI_Sendrecv(  
            localGrid[1], N, MPI_INT, rank - 1, 0,  
            localGrid[0], N, MPI_INT, rank - 1, 1,  
            MPI_COMM_WORLD,  
            MPI_STATUS_IGNORE  
        );  
    }  
  
    if(rank < size - 1) {  
        MPI_Sendrecv(  
            localGrid[rows], N, MPI_INT, rank + 1, 1,  
            localGrid[rows + 1], N, MPI_INT, rank + 1, 0,  
            MPI_COMM_WORLD,  
            MPI_STATUS_IGNORE  
        );  
    }  
}
```

FIGURE 3 – Extrait du code de la version MPI avec Halo Exchange

3.3.4. Résultats obtenus

Les tests ont été réalisés avec :

✧ Taille de grille : 1000×1000

✧ Nombre d'itérations : 100

Nombre de processus	Temps (ms)	Speedup	Efficacité
2 processus	809	$\frac{1986}{809} \approx 2,45$	$\frac{2,45}{2} \approx 122,5\%$
4 processus	520	$\frac{1986}{520} \approx 3,82$	$\frac{3,82}{4} \approx 95,5\%$

TABLE 4 – Performances de la version MPI avec speedup et efficacité

3.3.5. Analyse

La version MPI améliore les performances grâce à la distribution des calculs entre plusieurs processus.

Avec 2 processus, l'efficacité dépasse 100% (122,5%), ce qui peut s'expliquer par une meilleure utilisation du cache mémoire lors du partitionnement de la grille.

L'ajout de 2 processus supplémentaires (4 processus au total) permet d'atteindre un temps de 520 ms avec un speedup de 3,82. L'efficacité reste élevée à 95,5%, ce qui indique que les communications entre processus restent limitées par rapport au gain de calcul.

3.4 Version CUDA

Pour obtenir des performances encore plus élevées, une version CUDA a été développée.

CUDA permet d'utiliser la carte graphique (GPU) afin d'exécuter un très grand nombre de threads en parallèle.

3.4.1. Principe

Chaque cellule de la grille est traitée par un thread GPU.

Les calculs sont exécutés massivement en parallèle sur les cœurs du GPU.

3.4.2. Environnement matériel — GPU Tesla T4

Les tests CUDA ont été réalisés sur le GPU mis à disposition par Google Colaboratory (relevé via `nvidia-smi`) :

Paramètre	Valeur
Modèle GPU	NVIDIA Tesla T4
Mémoire GPU	15 360 MiB
Driver NVIDIA	580.82.07
Version CUDA (runtime)	13.0
Compilateur nvcc	11.5.119
TDP	70 W
Cœurs CUDA	2560

TABLE 5 – Caractéristiques du GPU Tesla T4 utilisé sur Colab

3.4.3. Techniques utilisées

- ✧ `__global__` : définition du kernel CUDA
- ✧ `cudaMalloc()` : allocation mémoire sur le GPU
- ✧ `cudaMemcpy()` : transfert des données CPU/GPU
- ✧ `cudaDeviceSynchronize()` : synchronisation des threads GPU
- ✧ Technique de **swap** entre les deux grilles
- ✧ **Grille de threads** : blocs de 16×16 threads, soit 62×62 blocs pour $N = 1000$

3.4.4. Fonctionnement

À chaque itération :

1. Les threads GPU calculent les nouvelles infections
2. Les cellules sont mises à jour en parallèle
3. Les grilles sont échangées avec la technique de swap

```

__global__ void update(int *grid, int *newGrid) {

    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    if(i > 0 && i < N-1 && j > 0 && j < N-1) {

        int idx = i * N + j;

        if(grid[idx] == 0) {

            int up    = grid[(i-1)*N + j];
            int down  = grid[(i+1)*N + j];
            int left  = grid[i*N + (j-1)];
            int right = grid[i*N + (j+1)];

            if(up == 1 || down == 1 || left == 1 || right == 1)
                newGrid[idx] = 1;
            else
                newGrid[idx] = 0;

        } else {
            newGrid[idx] = grid[idx];
        }
    }
}

```

FIGURE 4 – Extrait du code de la version CUDA

3.4.5. Résultat obtenu

Les tests ont été réalisés avec :

- ✧ Taille de grille : 1000×1000
- ✧ Nombre d'itérations : 100

Le temps d'exécution obtenu est :

Temps (ms)	Speedup	Efficacité
138	$\frac{1986}{138} \approx 14,39$	– (GPU : milliers de cœurs)

TABLE 6 – Performance de la version CUDA

3.4.6. Analyse

La version CUDA offre les meilleures performances parmi toutes les implémentations réalisées.

Le GPU permet l'exécution simultanée de milliers de threads, ce qui accélère fortement les calculs de propagation.

La parallélisation massive du GPU réduit considérablement le temps d'exécution par rapport aux versions séquentielle, OpenMP et MPI. L'efficacité n'est pas calculée de façon classique pour CUDA car le GPU dispose de plusieurs milliers de cœurs CUDA dont le nombre exact dépend du matériel utilisé.

4. Comparaison des performances

Les différentes implémentations ont été testées avec les paramètres suivants :

✧ Taille de la grille : 1000×1000

✧ Nombre d'itérations : 100

4.1 Rappel des formules

$$\text{Speedup} = \frac{T_{\text{séquentiel}}}{T_{\text{parallèle}}}$$

$$\text{Efficacité} = \frac{\text{Speedup}}{p} \times 100\%$$

où $T_{\text{séquentiel}} = 1986$ ms et p = nombre de processeurs/threads.

4.2 Tableau comparatif

Version	Temps (ms)	Speedup	Efficacité
Séquentielle	1986	1,00	100% (référence)
OpenMP (2 threads)	1048	1,89	94,5%
OpenMP (4 threads)	939	2,11	52,8%
MPI (2 processus)	809	2,45	122,5%
MPI (4 processus)	520	3,82	95,5%
CUDA	138	14,39	–

TABLE 7 – Comparaison complète des performances

4.3 Analyse des résultats

Les résultats montrent une amélioration progressive des performances grâce aux différentes techniques de parallélisation.

- ✧ La version **séquentielle** est la plus lente car tous les calculs sont exécutés sur un seul processeur. Elle sert de référence avec un speedup de 1.
- ✧ La version **OpenMP** réduit le temps d'exécution grâce à l'utilisation de plusieurs threads. Avec 4 threads, l'efficacité descend à 52,8%, ce qui traduit un surcoût de synchronisation entre threads.
- ✧ La version **MPI** améliore davantage les performances en répartissant le calcul entre plusieurs processus indépendants. L'efficacité élevée (95,5% à 4 processus) confirme que les communications MPI restent peu coûteuses pour ce problème.
- ✧ La version **CUDA** offre les meilleures performances avec un speedup de 14,39, grâce à l'exécution simultanée de milliers de threads GPU.

Le GPU permet de traiter simultanément un très grand nombre de cellules, ce qui explique la forte réduction du temps d'exécution.

4.4 Variation de la taille de la grille ($ITER = 100$ fixé)

On a testé les implémentations séquentielle et OpenMP pour $N \in \{500, 1000, 2000\}$ afin d'observer comment le temps d'exécution évolue avec la taille du problème.

Version	N = 500 (ms)	N = 1000 (ms)	N = 2000 (ms)
Séquentielle	371	1986	4057
OpenMP (2 threads)	249	1048	5435
OpenMP (4 threads)	273	939	4140

TABLE 8 – Temps d'exécution en fonction de la taille de grille N ($ITER = 100$)

Observations :

- ✧ Le temps séquentiel croît approximativement en $O(N^2)$ avec la taille de la grille.
- ✧ Pour $N = 2000$, OpenMP 2 threads est plus lent que la version séquentielle ($5435 > 4057$ ms), ce qui s'explique par un surcoût de synchronisation et une pression mémoire accrue dépassant la capacité du cache partagé.
- ✧ OpenMP 4 threads reste légèrement plus rapide que le séquentiel à grande taille ($N = 2000$: 4140 ms vs 4057 ms), mais le gain est très limité, illustrant les limites de la mémoire partagée pour de larges grilles.

4.5 Variation du nombre d'itérations ($N = 1000$ fixé)

On mesure l'impact du nombre d'itérations sur le temps d'exécution de la version séquentielle.

Version	ITER = 50 (ms)	ITER = 100 (ms)	ITER = 200 (ms)
Séquentielle	488	1986	1877

TABLE 9 – Temps d'exécution séquentiel en fonction du nombre d'itérations ($N = 1000$)

Observation : Le temps pour $ITER = 200$ (1877 ms) est inférieur à celui de $ITER = 100$ (1986 ms). Cela s'explique par le fait qu'après un certain nombre d'itérations, la propagation atteint les bords de la grille et la quasi-totalité des cellules sont déjà infectées. Les calculs deviennent alors triviaux (aucun changement d'état), réduisant le temps moyen par itération.

4.6 Décomposition du temps CUDA : transferts vs calcul kernel

La version CUDA détaillée mesure séparément les trois phases du traitement GPU :

1. **Transfert H2D** (Host $\rightarrow$ Device) : copie de la grille initiale vers la mémoire GPU
2. **Calcul kernel** : exécution des 100 itérations de propagation sur le GPU
3. **Transfert D2H** (Device $\rightarrow$ Host) : rapatriement du résultat final vers le CPU

Phase	Temps (ms)	Part du total
Transfert CPU $\rightarrow$ GPU (H2D)	≈ 5	$\approx 3,6\%$
Calcul kernel GPU	≈ 128	$\approx 92,8\%$
Transfert GPU $\rightarrow$ CPU (D2H)	≈ 5	$\approx 3,6\%$
Total	138	100%

TABLE 10 – Décomposition du temps d'exécution CUDA ($N = 1000$, $ITER = 100$, Tesla T4)

Analyse : Le calcul kernel représente plus de 92% du temps total, ce qui confirme que les transferts mémoire CPU $\leftrightarrow$ GPU sont négligeables pour ce problème ($< 4\%$ du temps total). Ce rapport calcul/transfert très favorable justifie pleinement l'utilisation du GPU pour ce type de simulation.

5. Conclusion

Dans ce projet, plusieurs versions d'une simulation de propagation d'épidémie sur une grille 2D ont été développées :

- ✧ Version séquentielle
- ✧ Version OpenMP
- ✧ Version MPI
- ✧ Version CUDA

L'objectif principal était d'étudier l'impact de la programmation parallèle sur les performances, mesuré à travers le **speedup** ($S = T_{\text{seq}}/T_{\text{par}}$) et l'**efficacité** ($E = S/p \times 100\%$).

Les résultats obtenus montrent clairement que les techniques de parallélisation permettent de réduire considérablement le temps d'exécution.

La version CUDA est la plus performante grâce à l'utilisation du GPU et du parallélisme massif (speedup de 14,39).

Ce travail montre l'importance des architectures parallèles pour résoudre efficacement des problèmes de grande taille.

